Mini Project 2: LLM-powered RESTful Web Service

1. Project Requirements

This project aims at implementing an LLM-powered RESTful web service for translating from a source language (e.g., English) to Moroccan Arabic Dialect (Darija). The RESTful endpoint is called **TranslatorResource** and it has a business method called **translate**(), which accepts a text (e.g., English) as a parameter, then returns its translation. The translation task is performed by an LLM (Large Language Model) such as Google Gemini (https://ai.google.dev/pricing#1_5flash) or other free-of-charge ones.

Your task is to implement the corresponding Java RESTful web service and some client applications in the following way:

- Implement the REST web service class using Jakarta RESTful Web Services API.
- Test your implementation using Postman, cURL, Thunder Client, or any other HTTP tool.
- Secure the RESTful web service endpoint using Jakarta Authentication Specification. This Specification can be used to set up various forms of authentication, including *basic*, *form-based*, *digest*, and *token-based* authentication. In this project, you can consider *basic authentication*.
- Build a Chrome extension using Manifest V3. Use *chrome.sidePanel API* to host content (i.e., call the REST endpoint) in the browser's side panel alongside the main content of a webpage to immediately perform a translation.
    - The selected text should be automatically copied and pasted.
    - The extension can be accessed and selected via the context (right-click) menu.
- Implement a PHP client app, Python app, and mobile app using React Native.
- Extension:
    - Add other features, such as speech translation (voice-to-voice translation), read aloud (i.e., text-to-speech voice reader), etc.
    - You may use a lightweight LLM for local deployment.
    - You may use an embedded container for deploying your REST endpoint.

2. Deliverable:

To complete the project, the following deliverables must be submitted:

- **Fully Functional Maven Project:** Source code for the RESTful web service application.
- **Fully Functional Projects:** Source code for the Chrome extension and the other clients.
- **Technical Architecture (UML):**
    - Optional: Use Case diagrams or Sequence diagrams to further clarify user interactions and message flow.
    - Class Diagram: Detailing the software structure and relationships between classes.
    - Deployment Diagram: Illustrating the physical network nodes (Server vs. Clients)
- **Demo Video:** A 5-minute video showcasing a walkthrough of the source code and a live demonstration of the application's features.
- **GitHub Link:** A shared link to your GitHub repository containing the full source code, commit history, and a descriptive README.md.
- **Optional Technical Post:** A link to a blog post (e.g., Medium, Dev.to, or a personal forum) detailing the development process, technical challenges, and final implementation.